
ARCHITECTURAL PRIMER & ENGINEERING GUIDE

Cert-Reflex

An autonomous, self-healing PKI lifecycle daemon

A resilient Java / Spring Boot service that monitors digital certificates across multiple live HTTPS endpoints, detects expiration and on-disk tampering, issues replacements through a secure cloud HSM/KMS boundary, and hot-swaps them into the running web server without dropping a single active connection.

JAVA · SPRING BOOT 3.2 · BOUNCY CASTLE · AWS KMS · POSTGRESQL · REACT

Design · Cryptography · High Availability · Distributed Safety

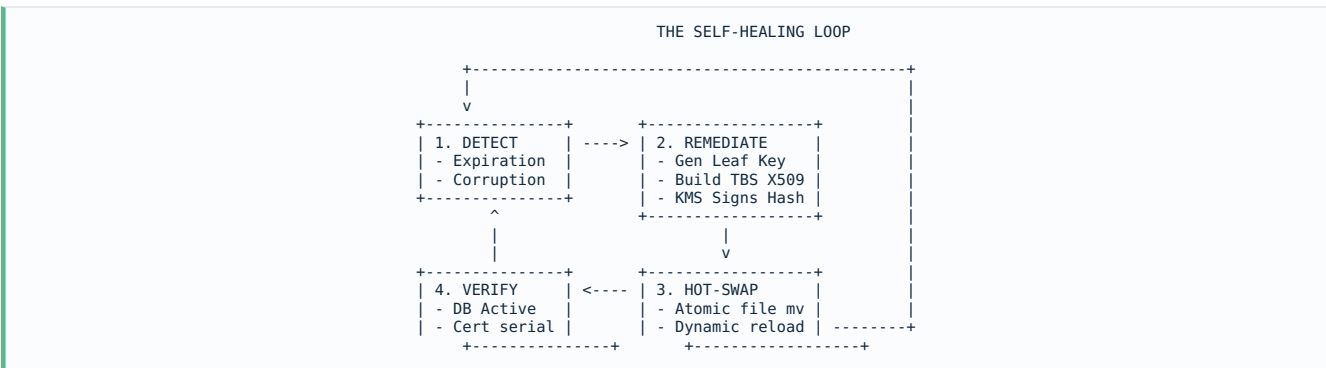
Contents

1	Executive Overview: What You Are Building
2	Why This Project Matters
3	The Zero-to-One Cryptography Cheat Sheet
4	Architectural Deep Dive, Section by Section
5	Technical Tradeoffs & Design Decisions
6	The Core Technical Interview Narrative
7	Concrete Implementation Checklist

1 · Executive Overview: What You Are Building

You are building an autonomous, self-healing public key infrastructure (PKI) lifecycle daemon called **Cert-Reflex**.

In plain terms: you are writing a small, resilient backend service in Java and Spring Boot that monitors digital certificates across multiple active web endpoints, detects when any certificate is either **nearing expiration** or has been **tampered with / corrupted on disk**, issues a fresh certificate through a secure cloud HSM/KMS boundary, and **hot-swaps it into the live web server memory without dropping a single active network connection**.



2 · Why This Project Matters

Companies like Keytos build enterprise software (such as EZCA, EZMonitor, and EZSSH) because managing digital certificates manually is one of the most failure-prone tasks in modern IT:

- **Human oversight causes outages.** Teams routinely forget to renew certificates. When an internal cert expires, microservices stop talking to one another, databases drop incoming queries, and customer-facing apps produce fatal connection errors.
- **Bad security practices are rampant.** When developers build custom PKI tooling, they often store the master Certificate Authority (CA) private key as an unencrypted file (`ca.key`) directly on disk. If that host is compromised, the entire corporate trust domain is broken.
- **Maintenance requires downtime.** Historically, installing a new certificate required running `systemctl restart nginx` or rebooting the application, dropping in-flight user requests.

By delivering Cert-Reflex, you prove you understand:

1. **Real-world security boundaries:** keeping private keys isolated in hardware/KMS.
2. **High availability (HA):** reloading cryptographic material without terminating client TCP connections.

3. **Resilience & distributed safety:** handling race conditions with distributed database locks and verifying file integrity before failures become catastrophic.

3 · The Zero-to-One Cryptography Cheat Sheet

You do not need deep mathematical knowledge of elliptic curves to build or explain this system. You need to understand how public key infrastructure functions as an identity protocol.

3.1 Asymmetric Key Pairs (Public vs. Private)

Public-key cryptography uses a mathematically linked pair of keys:

- **Private Key:** a secret string of numbers that must never leave its secure boundary. It is used to generate digital signatures or decrypt data.
- **Public Key:** derived from the private key, but mathematically safe to distribute to anyone. It is used to verify signatures or encrypt data for the private key holder.
- **Analogy:** the private key is a custom physical wax seal; the public key is a reference catalog distributed globally that allows anyone to inspect an impression and verify that it came from that specific seal.

3.2 X.509 Certificate

An X.509 certificate is an internationally standardized digital document (RFC 5280) that binds an identity (e.g. `demo-a.internal`) to a specific public key. It contains:

- **Subject / Common Name (CN):** who this certificate belongs to.
- **Subject Alternative Name (SAN):** modern browsers and clients ignore the CN and require SANs (e.g. `DNS:localhost, IP:127.0.0.1`).
- **Validity Window:** `notBefore` (start time) and `notAfter` (expiration timestamp).
- **Serial Number:** a unique integer identifying this certificate instance.
- **Public Key:** the public key of the service hosting the website/API.
- **CA Digital Signature:** the cryptographic stamp from a Certificate Authority verifying the contents.

3.3 Certificate Authority (CA)

A trusted entity whose public key is pre-installed in trust stores (operating systems, browsers, Java runtimes). When a client connects to `demo-a`, `demo-a` presents its certificate. The client inspects the signature on that certificate; because the client trusts the CA that signed it, the client trusts that `demo-a` is legitimate.

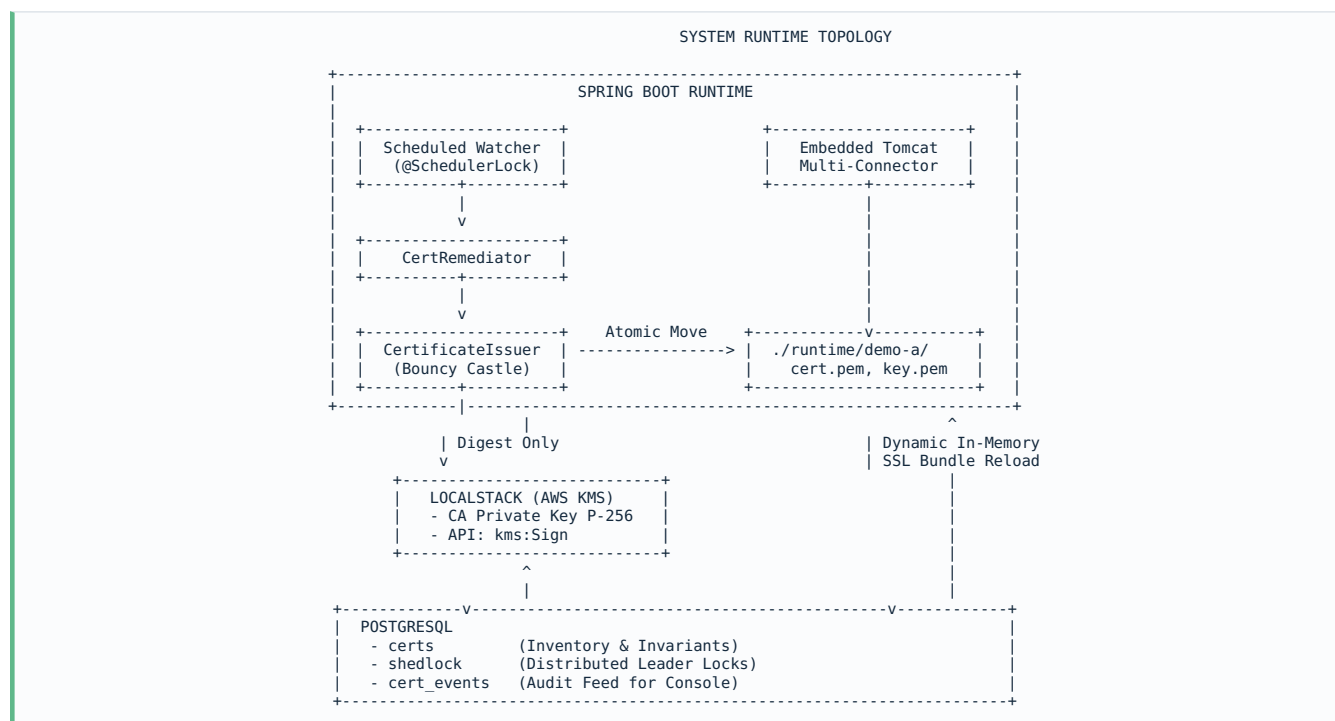
3.4 TBS Certificate ("To Be Signed")

Before a certificate is signed, it is assembled as a raw data structure known as the **TBSCertificate**. It contains all the fields (Subject, SAN, Public Key, Validity) except the final signature. The signer takes a cryptographic hash (SHA-256) of this binary structure, encrypts that hash using its private key, and appends the resulting signature to form the completed X.509 certificate.

3.5 ECDSA vs. RSA

- **RSA:** the traditional public key algorithm, relying on the difficulty of factoring large prime numbers. RSA keys are large (typically 2048 or 4096 bits) and computationally heavier.
- **ECDSA:** a modern standard based on elliptic curves over finite fields. A **P-256** (secp256r1) EC key offers security equivalent to a 3072-bit RSA key, but with vastly smaller key sizes and significantly faster signature generation.

4 · Architectural Deep Dive, Section by Section



4.1 The Issuer (CA) & The KMS Boundary

The Design

The CA's signing private key is created directly inside **AWS KMS** (emulated via LocalStack) with the specification `ECC_NIST_P256` and key usage `SIGN_VERIFY`.

KEY LIFECYCLE RULE

The CA private key material is never extracted, downloaded, or held in application memory.

How Remote Signing Works

Normally, Bouncy Castle uses a local private key to sign certificates. Because the private key is trapped inside KMS, you implement a custom `org.bouncycastle.operator.ContentSigner`:

1. The leaf node generates its own ephemeral EC P-256 key pair locally — this is the key that will terminate TLS traffic on Tomcat.
2. `CertificateBuilder` constructs the **TBS (To Be Signed)** structure containing the domain, validity period, and the leaf public key.
3. The custom `KmsContentSigner`:
 - calculates the SHA-256 hash of the TBS bytes locally;
 - dispatches an HTTP API call to KMS — `kmsClient.sign(...)` with `MessageType: DIGEST` and `SigningAlgorithm: ECDSA_SHA_256`;
 - KMS executes the math on the digest inside its secured perimeter and returns an ASN.1 DER-encoded signature.
4. Bouncy Castle stitches the returned signature onto the TBS structure, producing a valid X.509 certificate PEM.

4.2 Inventory (PostgreSQL)

The PostgreSQL database serves as the source of truth for the certificate fleet.

```

CREATE TABLE certs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  service_name TEXT NOT NULL,
  common_name TEXT NOT NULL,
  serial_number NUMERIC(39,0) NOT NULL,
  not_before   TIMESTAMPTZ NOT NULL,
  not_after    TIMESTAMPTZ NOT NULL,
  status       TEXT NOT NULL DEFAULT 'ACTIVE',
  -- ACTIVE, EXPIRING, CORRUPTED, ROTATING, FAILED
  cert_pem     TEXT NOT NULL,
  cert_path    TEXT NOT NULL,
  key_path     TEXT NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Crucial distributed invariant:
CREATE UNIQUE INDEX idx_certs_service_active
  ON certs(service_name)
  WHERE status = 'ACTIVE';

```

Why the Partial Index Matters

The index `WHERE status = 'ACTIVE'` guarantees at the storage engine level that **no service can ever have more than one active certificate registered at the same time**. During rotation, the state machine transitions the previous certificate out of `ACTIVE` before elevating the newly issued certificate to `ACTIVE`, making split-brain states impossible.

4.3 The Watcher

The watcher is a scheduled task running every 5 seconds. In each cycle, it executes two distinct health checks.

1. Temporal expiration check:

```

SELECT * FROM certs
WHERE status = 'ACTIVE'
  AND not_after - now() < INTERVAL '30 seconds';

```

2. Disk integrity & parity check:

- Reads the on-disk file at `cert_path`.
- Verifies the file parses cleanly as an X.509 certificate (detecting file truncation or garbage bytes).
- Verifies that the public key of the file on disk matches the public key in the database record (detecting unauthorized out-of-band certificate swaps).

Distributed Concurrency with ShedLock

If you scale the service to multiple containers, all instances would run the 5-second scheduled job simultaneously, risking duplicate renewals and duplicate KMS charges.

```

@Scheduled(fixedDelay = 5000)
@SchedulerLock(name = "CertWatcher_scan",
  lockAtLeastFor = "PT2S",
  lockAtMostFor = "PT4S")
public void runCertScan() {
  // Only one instance across the entire cluster can run this at a time
}

```

ShedLock writes a row to a `shedlock` database table. The instance that acquires the lock runs the scan; all other instances skip the tick.

4.4 The Remediator & Zero-Downtime Hot-Swap

Once an anomaly is detected:

1. The remediator marks the database status as `ROTATING`.
2. It requests a new certificate from the `CertificateIssuer`.
3. **Atomic file write (crucial reliability engineering)**. Writing directly to `cert.pem` introduces a race condition: if Spring Boot's file watcher reads the file while it is half-written, it will attempt to parse incomplete PEM bytes and

crash the SSL context. The remediator instead writes to a temporary file in the same directory (`cert.pem.tmp`) and performs an atomic filesystem rename:

```
Files.move(
    tempPath,
    targetPath,
    StandardCopyOption.ATOMIC_MOVE,
    StandardCopyOption.REPLACE_EXISTING
);
```

4. **Dynamic reload.** Spring Boot 3.2+ features built-in SSL bundle reloading (`reload-on-update: true`). It detects the atomic file change, initializes a new `SSLContext` with the fresh key pair, and swaps the pointer inside the running embedded Tomcat connector.
5. **Zero disruption guarantee.** Established TCP/TLS connections continue communicating over their existing symmetric session keys. Any incoming TLS handshake immediately receives the new certificate. Not a single connection is reset.

4.5 Demo Multi-Connector Setup

Rather than spinning up four independent Spring Boot services (which inflates Docker memory usage), a single Spring Boot application registers multiple Tomcat connectors on different ports:

- Port `8080` — Management & Admin API (plain HTTP).
- Port `8443` — `demo-a` (HTTPS, SSL Bundle A).
- Port `8444` — `demo-b` (HTTPS, SSL Bundle B).
- Port `8445` — `demo-c` (HTTPS, SSL Bundle C).

```
@Configuration
public class MultiConnectorConfig {
    @Bean
    public WebServerFactoryCustomizer<TomcatServletWebServerFactory> connectorCustomizer() {
        return factory -> {
            factory.addAdditionalTomcatConnectors(
                createConnector("demo-a", 8443),
                createConnector("demo-b", 8444),
                createConnector("demo-c", 8445)
            );
        };
    }
}
```

4.6 Failure Injection Engine

To demonstrate self-healing on demand, an internal endpoint allows injecting failures:

- `POST /internal/failures/demo-a?type=EXPIRE` — updates the `not_after` column in PostgreSQL to `now() - INTERVAL '10 seconds'`. On the next 5-second tick, the watcher flags it as `EXPIRING` and triggers rotation.
- `POST /internal/failures/demo-a?type=CORRUPT` — overwrites `runtime/demo-a/cert.pem` with random garbage bytes. The watcher fails to parse the file, flags it as `CORRUPTED`, generates a new key pair and certificate, and replaces the damaged file.

4.7 The Operations Console UI

A minimalist React dashboard polling `/internal/certs` and `/internal/events` every 1 second:

- Shows 3–4 listener cards side-by-side with real-time countdown clocks.
- Border status colors:
 - **Green (#5FB88A)** — active and healthy.
 - **Yellow (#E0A23D)** — expiring (< 30 seconds remaining).
 - **Blue (#4A9DE0)** — rotating (issuance in progress).
 - **Red (#E0623D)** — corrupted / failed.
- One-click **Expire** and **Corrupt** buttons for interactive demos.

- Terminal-style scrolling event log showing state transitions as they occur.
- The React app is pre-built (`npm run build`) and its assets committed into `src/main/resources/static/dashboard/` , ensuring reviewers don't need Node.js installed to view the demo.

5 · Technical Tradeoffs & Design Decisions

When discussing this project in an interview, explaining **why** you chose specific designs — and what trade-offs they entail — demonstrates engineering maturity.

Decision	What Was Chosen	Alternative Rejected	Why This Choice Was Made
Key Custody Model	CA key in KMS; leaf keys generated locally	Storing CA private key in a local <code>.pem</code> file	Storing the CA key on disk creates a catastrophic single point of failure. Keeping it in KMS demonstrates understanding of real-world key custody.
Emulation Strategy	LocalStack Community Edition	Real AWS KMS account	Eliminates cloud costs and complex AWS credential provisioning for anyone reviewing or running the project locally.
Signing Mechanism	Custom <code>ContentSigner</code> sending <code>DIGEST</code>	Passing entire cert bytes to KMS, or using the OpenSSL CLI	KMS asymmetric signing APIs have payload size limits and are network operations; hashing locally and signing the 32-byte digest is standard, efficient cryptographic practice.
Concurrency Model	ShedLock with a PostgreSQL table	Redis / distributed consensus (Raft, ZooKeeper)	Introducing Redis adds unnecessary operational overhead for a weekend build. PostgreSQL is already in the stack; using it for distributed locking keeps the architecture lean.
Hot-Swap Engine	Spring Boot 3.2 SSL bundles	Nginx sidecar or Envoy proxy restart	Native JVM reloading avoids external proxy sidecars, allowing the application to manage its own cryptographic lifecycle end-to-end.
Process Topology	Single process with multi-connector ports	Four separate microservice containers	Multi-container setups consume substantial RAM and add orchestration complexity without demonstrating additional PKI principles.

6 · The Core Technical Interview Narrative

6.1 The Elevator Pitch

"I built Cert-Reflex, a self-healing PKI lifecycle engine in Java and Spring Boot. It continuously monitors running HTTPS endpoints, detects expiring or corrupted certificates, and executes zero-downtime hot-swaps. The core design principle was strict key custody: the intermediate CA private key never enters application memory — signing takes place over an API boundary using an asymmetric EC key in KMS. When a certificate requires renewal, the system generates a local leaf key, builds the X.509 structure, obtains the remote signature, and performs an atomic filesystem swap that Spring Boot's SSL engine reloads without terminating in-flight connections."

6.2 High-Yield Interview Questions & Answers

Q: "Why didn't you generate the leaf private key inside KMS as well?"

The CA private key and leaf private keys have completely different operational lifecycles. A CA private key signs occasionally, possesses high organizational trust, and must be guarded inside a hardware boundary. A leaf private key is used constantly to terminate incoming TLS handshakes at microsecond latencies. If every TLS handshake required an API roundtrip to KMS, network latency would bring the service to a crawl. Leaf keys must reside in memory on the termination host.

Q: "How do you prove the hot-swap actually caused zero downtime?"

In the demo verification suite, I run a persistent HTTP client executing consecutive requests against the endpoint while injecting an expiration fault. The client script captures the server certificate serial number on every response. We observe the serial number update in real time across requests, with zero dropped packets, zero connection resets, and no TCP reconnect penalties.

Q: "What happens if two instances try to rotate a certificate at the exact same time?"

We defend against this at two layers. First, the watcher job is guarded by ShedLock, which uses PostgreSQL row-level distributed locking to ensure only one replica can trigger a rotation workflow during a given window. Second, the schema enforces a partial unique index on `(service_name) WHERE status = 'ACTIVE'`. Even if a race condition bypassed the application layer, the database engine would reject duplicate active certificates.

Q: "Why use atomic file operations during the swap?"

Spring Boot's SSL bundle reload relies on filesystem notification watchers. If you overwrite a file via a standard stream, there is a microsecond window where the file is partially written or empty. If the file watcher fires during that window, the server attempts to parse an incomplete PEM structure, throwing an `SSLException` and corrupting the active context. Writing to a `.tmp` file and executing an `ATOMIC_MOVE` guarantees that the file inode swap happens instantaneously at the OS level.

7 · Concrete Implementation Checklist

Follow this phased approach to build the service systematically.

Phase 1 · The Cryptographic Core

- ☐ Set up `docker-compose.yml` with PostgreSQL and LocalStack (configured for KMS).
- ☐ Write a bootstrap script (`create-ca-key.sh`) using `awslocal kms create-key --key-usage SIGN_VERIFY --key-spec ECC_NIST_P256`.
- ☐ Implement `KmsContentSigner.java` :
 - extend Bouncy Castle's `ContentSigner` ;
 - override `getSignature()` to call `KmsClient.sign()` with `MessageType.DIGEST`.
- ☐ Implement `CertificateIssuer.java` :
 - generate an ephemeral EC P-256 leaf key pair;
 - build the X.509 structure with SAN extensions using `JcaX509v3CertificateBuilder`.
- ☐ Write `CertificateIssuerIT.java` using Testcontainers to verify the issued certificate parses and validates against the CA public key.

Phase 2 · Live Listener & Hot-Swap

- ☐ Configure `application.yml` with `spring.ssl.bundle.pem` pointing to `runtime/demo-a/cert.pem` with `reload-on-update: true`.
- ☐ Implement `MultiConnectorConfig.java` to spin up Tomcat connectors across ports 8443, 8444, and 8445.
- ☐ Implement the atomic file write utility in `CertRemediator.java`.
- ☐ Validate hot-reloading: curl the HTTPS port, overwrite the files using atomic move, curl again, and confirm the new certificate serial is returned.

Phase 3 · Watcher & State Engine

- ☐ Create Flyway or SQL scripts for the `certs`, `cert_events`, and `shedlock` tables.
- ☐ Implement `CertWatcherJob.java` using `@Scheduled` and `@SchedulerLock`.
- ☐ Add the 30-second expiration query.
- ☐ Add the disk corruption check: parse `cert.pem` from disk using Bouncy Castle's `PEMParser`, comparing its public key with the database record.

Phase 4 · Failure Injection & Admin APIs

- ☐ Implement `AdminController.java` :
 - `GET /internal/certs`
 - `GET /internal/events`
 - `POST /internal/failures/{serviceName}?type=EXPIRE`
 - `POST /internal/failures/{serviceName}?type=CORRUPT`
- ☐ Write `inject-failure.sh` to trigger failures via terminal.

Phase 5 · UI Dashboard & Demonstration Video

- ☐ Build the single-page React console with Vite (listener cards, timers, event logs, action buttons).
- ☐ Run `npm run build` and output static assets to `src/main/resources/static/dashboard/`.
- ☐ Test the end-to-end flow: run `docker compose up`, trigger failures from the UI, and observe real-time remediation.

- Record a clean 30-second screen capture demonstrating an injected failure, the console state change, and the zero-downtime certificate transition. Include this capture in your repository README.